Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: https://github.com/Ashwath-Raj/CSE-204-DAA-assignments-

1. Greedy Approach: Implementation of Huffman Code

Problem

Assume the numbers represent counts of letters in hundreds from a file: a:20, c:11, d:2, e:10, o:15, m:8, s:10, t:22, u:2. Construct an optimal Huffman code, show the tree and every merge step, give the encoding for each letter, and calculate the memory required for the encoded file.

Method

Repeatedly combine the two nodes with the smallest frequencies. Insert their sum as a new parent node and continue until one tree remains. Assign 0 to each left edge and 1 to each right edge. Equal-frequency ties are resolved by insertion order, as in the source program.

Output:

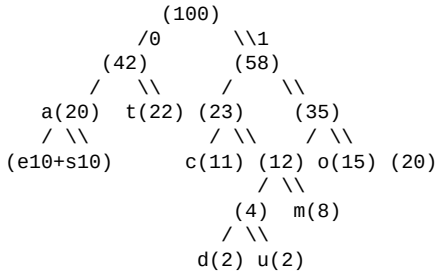
```
$ ./huffman_lab8
Huffman Merge Steps:
2 + 2 = 4
4 + 8 = 12
10 + 10 = 20
11 + 12 = 23
15 + 20 = 35
20 + 22 = 42
23 + 35 = 58
42 + 58 = 100
Huffman Codes:
e : 000
s : 001
t : 01
c : 100
d : 10100
u : 10101
m : 1011
o : 110
a : 111
```

Greedy merge steps

```
2 + 2 = 4
4 + 8 = 12
10 + 10 = 20
```

```
11 + 12 = 23
15 + 20 = 35
20 + 22 = 42
23 + 35 = 58
42 + 58 = 100
```

Huffman tree (left edge = 0, right edge = 1)



Huffman encoding table

Letter	Frequency	Huffman code	Bits per occurrence
a	20 × 100	111	3
c	11 × 100	100	3
d	2 × 100	10100	5
e	10 × 100	000	3
o	15 × 100	110	3
m	8 × 100	1011	4
s	10 × 100	001	3
t	22 × 100	01	2
u	2 × 100	10101	5

Memory calculation

Each frequency represents 100 occurrences. Total encoded size = 100 × [(20×3) + (11×3) + (2×5) + (10×3) + (15×3) + (8×4) + (10×3) + (22×2) + (2×5)] = 29,400 bits. In bytes: 29,400 ÷ 8 = 3,675 bytes. The result excludes file/header overhead.

Code:

```
/*
Program Name: Huffman Coding Using Greedy Approach
Program Description: Builds an optimal Huffman tree, prints codes and calculates encoded size.
Program Creation Time: Oct 17th, 2026
Program Creator: Ashwath Raj | AP25110010712
*/
#include<iostream>
#include<queue>
#include<vector>
#include<string>
using namespace std;

struct Node{
    char ch;
    int freq,order;
    Node *left,*right;
    Node(char c,int f,int o):ch(c),freq(f),order(o),left(nullptr),right(nullptr){}
};
struct Compare{
    bool operator()(Node *a,Node *b)const{
        if(a->freq!=b->freq) return a->freq>b->freq;
        return a->order>b->order;
    }
};
void codes(Node *p,string s,int &bits){
    if(!p->left&&!p->right){
        cout<<p->ch<<" : "<<s<<endl;
        bits+=p->freq*(int)s.size();
        return;
    }
    codes(p->left,s+"0",bits);
    codes(p->right,s+"1",bits);
}
int main(void){
    char ch[]={'a','c','d','e','o','m','s','t','u'};
    int f[]={20,11,2,10,15,8,10,22,2},order=0,bits=0;
    priority_queue<Node*,vector<Node*>,Compare> q;
    for(int i=0;i<9;i++) q.push(new Node(ch[i],f[i],order++));
    cout<<"Huffman Merge Steps:"<<endl;
    while(q.size()>1){
        Node *l=q.top();q.pop();
        Node *r=q.top();q.pop();
        cout<<l->freq<<" + "<<r->freq<<" = "<<l->freq+r->freq<<endl;
```

```
            Node *p=new Node('$',l->freq+r->freq,order++);
            p->left=l;p->right=r;q.push(p);
        }
        cout<<"Huffman Codes:"<<endl;
        codes(q.top(),"",bits);
        cout<<"Encoded size: "<<bits*100<<" bits ("<<bits*100/8<<" bytes)"<<endl;
        return 0;
    }
```

Complexity

Time Complexity: O(n log n), where n is the number of distinct symbols. Space Complexity: O(n) for the Huffman tree and priority queue.